

JOB CARD & VISIT TRACKING SYSTEM

STEP 1: BACKEND PROJECT SETUP

Complete Technical Documentation

Project:	Job Card & Visit Tracking System
Developer:	Arthur (animex)
Date Started:	February 2026
Step:	1 - Backend Foundation & Express Server Setup
Technology:	Node.js v20, Express.js 4.x, npm
Status:	✓ COMPLETED - Server Running Successfully
Files Created:	5 core files + package configuration

TABLE OF CONTENTS

Section	Description	Page
1	Executive Summary & Project Overview	3
2	Project Initialization & Directory Structure	5
3	Package.json Configuration	8
4	Environment Variables Setup (.env)	11
5	Database Configuration (database.js)	13
6	Express Server Implementation (server.js)	16
7	Middleware Architecture	22
8	Error Handling Implementation	25
9	Testing & Verification	27
10	Lessons Learned & Best Practices	29

1. EXECUTIVE SUMMARY & PROJECT OVERVIEW

Step 1 established the foundational infrastructure for the Job Card & Visit Tracking System backend. This critical first step involved initializing a Node.js project, installing essential dependencies, configuring the development environment, and creating a fully functional Express.js server with proper error handling, middleware architecture, and database connectivity. **Project Purpose:**

The Job Card & Visit Tracking System is designed to manage technician field visits, track job assignments, and maintain customer records. The backend serves as the API layer that will eventually support a frontend application, providing RESTful endpoints for user authentication, job card management, customer data, and visit tracking. **Step 1 Objectives:**

The primary goals of Step 1 were to:

- Initialize a professional Node.js project structure with appropriate organization
- Install and configure all necessary dependencies (Express, PostgreSQL client, security packages)
- Set up environment variable management for secure configuration
- Create database connection pooling for efficient PostgreSQL connectivity
- Implement a production-ready Express server with comprehensive middleware
- Establish error handling patterns that will scale as the application grows
- Create health check endpoints for monitoring and deployment verification
- Document the setup process for team collaboration and future reference

Key Achievements:

- **Professional Project Structure:** Created organized directory hierarchy (src/config, src/routes, src/controllers, src/services, src/middleware) following industry best practices for separation of concerns.
- **Dependency Management:** Installed 8 core packages including Express.js for the web framework, pg for PostgreSQL connectivity, dotenv for environment management, helmet for security headers, cors for cross-origin resource sharing, bcrypt for password hashing, jsonwebtoken for authentication, and joi for validation.
- **Environment Configuration:** Implemented secure configuration management using .env files, separating sensitive data (database credentials, JWT secrets) from source code, and added .env to .gitignore for security.
- **Database Connection Pooling:** Configured PostgreSQL connection pool with optimized settings (max 20 connections, 30-second idle timeout, 2-second connection timeout) for efficient database resource utilization.
- **Express Server Architecture:** Built comprehensive server with middleware stack (helmet for security, cors for API access, json parser for request bodies, custom logger for debugging) and proper error handling.
- **Testing Infrastructure:** Created health check endpoint (/api/v1/health) and database test endpoint (/api/v1/db-test) enabling verification of server and database connectivity.
- **Production Readiness:** Implemented graceful shutdown handlers (SIGINT, SIGTERM) ensuring clean server termination and database connection closure when deployment environments restart containers.

2. PROJECT INITIALIZATION & DIRECTORY STRUCTURE

2.1 Initial Setup Commands

The project initialization began with creating the directory structure and initializing npm (Node Package Manager). This process establishes the project as a Node.js application and creates the package.json file that tracks dependencies and project metadata.

```
# Navigate to project workspace
cd ~/JobCardSystem

# Create backend directory
mkdir backend
cd backend

# Initialize npm with default settings
npm init -y

# Create directory structure
mkdir -p src/config src/routes src/controllers src/services src/middleware src/utils src/validators
```

2.2 Directory Structure Explanation

The directory structure follows the Model-View-Controller (MVC) pattern adapted for API development. Each directory serves a specific purpose in organizing code by responsibility:

Directory	Purpose	Contents in Future Steps
src/config/	Configuration files	database.js (PostgreSQL connection pool)
src/routes/	API endpoint definitions	auth.routes.js, user.routes.js, customer.routes.js, jobcard.routes.js
src/controllers/	HTTP request handlers	auth.controller.js, user.controller.js, etc.
src/services/	Business logic & DB queries	auth.service.js, user.service.js, etc.
src/middleware/	Reusable middleware	authenticate.js, authorize.js, validateRequest.js
src/utils/	Utility functions	authHelpers.js (bcrypt, JWT functions)
src/validators/	Input validation schemas	auth.validator.js, user.validator.js, etc.

Why This Structure?

This organization provides several critical benefits: **Separation of Concerns:** Each directory handles one type of responsibility. Routes define endpoints, controllers handle HTTP, services contain business logic, middleware handles cross-cutting concerns. This makes code easier to test, modify, and understand. **Scalability:** As the application grows, new features are added by creating new files in existing directories rather than creating new directories. For example, when adding customer management, we add customer.routes.js, customer.controller.js, and customer.service.js without restructuring. **Testability:** Pure business logic in services can be unit tested independently of HTTP concerns. Middleware can be tested in isolation. Controllers can be tested with mocked services. **Team Collaboration:** Multiple developers can work on different features simultaneously with minimal merge conflicts. One developer works on user management in user.*.js files while another works on job cards in jobcard.*.js files. **Industry Standard:** This structure is widely recognized in the

Node.js/Express community, making it easier for new team members to understand the codebase and contributing to maintainability.

3. PACKAGE.JSON CONFIGURATION

3.1 Installing Dependencies

Dependencies are third-party libraries that provide functionality our application needs. Rather than writing every feature from scratch, we leverage well-tested packages maintained by the community.

```
# Install production dependencies
npm install express pg dotenv helmet cors bcrypt jsonwebtoken joi

# Install development dependency
npm install --save-dev nodemon
```

3.2 Dependency Analysis

Package	Version	Purpose	Why We Need It
express	~4.18.x	Web framework	Handles HTTP routing, middleware, request/response
pg	~8.11.x	PostgreSQL client	Connects to database, executes queries, manages connections
dotenv	~16.3.x	Env management	Loads .env file into process.env for configuration
helmet	~7.1.x	Security headers	Sets HTTP headers protecting against common attacks
cors	~2.8.x	Cross-origin	Allows frontend on different domain to call API
bcrypt	~5.1.x	Password hashing	Securely hashes passwords, prevents plaintext storage
jsonwebtoken	~9.0.x	JWT tokens	Creates and verifies authentication tokens
joi	~17.11.x	Validation	Validates request data with schemas and error messages
nodemon	~3.0.x	Auto-restart (dev)	Watches files and restarts server on changes